CS781 Capstone Project: In-House Legal Intelligence System

Granth Choudhary
250397
granthc25@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)
https://github.com/GranthChoudharyismyname/CS781-Capstone$_{P}roject$

Abstract

Law firms cannot send client data to third-party LLMs, so they need a locally hosted legal language model that supports everyday legal NLP tasks. I built a seven-phase pipeline covering language modelling, entity recognition, rhetorical roles, judgment prediction, statute identification, prior-case retrieval, and summarization. My primary Phase-1 foundation is a 115M parameters decoder-only Transformer (12 layers, RoPE, gated attention) with a 30K BPE vocabulary, trained on about 400M tokens of Acts, Supreme Court, High Court, and Tribunal text, reaching validation perplexity 7.32. This base model was fine tuned with multiple PEFT methods in the subsequent phases and an almost consistent result throughout phases was that full fine tuning outperformed most PEFT methods.

1 Competition Result

Codabench username: g250397 **Roll number:** 250397

Table 1: Official phase scores and ranks.

Phase	Task	Primary metric (value)	Rank	Valid submissions
1	Legal LM Pretraining	0.8346	41	6
2	Legal NER	0.5234	66	9
3	Rhetorical role prediction	0.4623	90	11
4	Court judgment prediction	0.758	40	9
5	Legal statute identification	0.2967	118	3
6	Abstractive summarization	0.8438	49	7
7	Prior Case Retrieval (PCR)	0.3674	79	7

Total valid Codabench submissions across all phases: 61
Date ranks were recorded: 9/11/2026

I have met the per-phase minimum for all phases except for one which is the 5th phase. I have a approved leave during that period as well, as during that phase I did not have sufficient internet connectivity as I was not in campus for a national model rocketry competition that I part took in. I have more than the required number of submissions for all other phases without considering submissions simply involving hyperparameter tuning of already existing submissions.

2 Introduction and Problem Description

Law firms cannot send confidential client data, privileged communications or unfiled case material to third-party LLMs. They therefore need locally hosted legal language models capable of handling tasks such as entity identification, rhetorical role prediction, judgment prediction, statute identification, case retrieval and summarization. In this project, I built such a system for Indian legal text by training a compact language model from scratch under limited computational resources and adapting it to multiple tasks.

The project consists of seven phases. Phase 1 is a language model that generates 128-word continuations from 512-word prompts, evaluated using BERTScore F1. Phase 2 performs legal NER, predicting character spans across 12 entity types, evaluated using strict macro-F1. Phase 3 assigns 13 rhetorical roles to sentences from Corporate Law and IT documents, while Phase 4 predicts binary court judgments. Phase 5 identifies applicable statutes. Both are evaluated using macro-F1. Phase 6 performs abstractive summarization using BERTScore F1, and Phase 7 retrieves the top 7 prior cases using micro-F1@7.

The Phase-1 model had to be trained from scratch with 50M–300M parameters, without pretrained weights. All experiments had to use free compute, requiring careful management of session limits and checkpoints. Phases 2–7 could only use Phase-1 weights for initialization. Additional challenges included exact span matching in NER, threshold selection for multi-label statute prediction and handling judgments much longer than the model's context window.

In Phase 1, I compared two data-mixing strategies and encoder-decoder against decoder-only architectures, with the latter performing better. I also implemented a gated attention variant that gave slightly better results, while an NPMI-IDF-based tokenizer merging strategy performed worse than frequency-based merging. In subsequent phases, I experimented with several PEFT methods, although full fine-tuning generally performed better. Semantic chunking was used in Phases 6 and 7, with recursive summarization also explored in Phase 6.

The final system consists of a decoder-only legal language model with gated attention and six task-specific fine-tuned versions. Due to limited computational resources and time, I generally assumed that improvements in the underlying architecture would translate to downstream performance, rather than exhaustively testing every possible combination.

3 Data Analysis and Preprocessing

3.1 Sources and official splits

The Phase-1 corpus was divided into four sources: Acts (10%), High Court judgments (40%), Supreme Court judgments (40%) and Tribunal documents (10%). This distribution was chosen because Acts contain relatively restricted and repetitive language, whereas High and Supreme Court judgments have more varied vocabulary and are more representative of the documents used in subsequent phases. Tribunals were given a smaller share because they cover more niche cases. I also tested a 20-35-35-10 distribution, but it barely changed the results.

3.2 Corpus statistics that actually informed design

- **Phase 1:** The corpus contains over 4.5B tokens, but due to computational limitations, I initially trained on approximately 160M tokens (159M training and 1M validation). One epoch took around 4 hours on Kaggle GPUs. I later trained another version on 400M tokens, which was used as an alternative backbone in subsequent phases. The 30K BPE tokenizer was used to tokenize the data, which was packed into blocks of 1024 tokens.
- **Phase 2:** The two annotated folds provided 70 labelled documents, which I split into 59 training and 11 validation documents, keeping 35 documents for testing. The task involved 12 entity types represented through 25 BIO tags. Since the labels were highly imbalanced, I used inverse-frequency loss weighting. The tokenizer's inability to represent certain character-level boundaries also limited the maximum achievable strict span-F1, with an upper bound of approximately 0.984 on validation.
- **Phase 3:** Each domain had 40 training, 5 development and 5 test documents. Corporate Law documents were longer than IT documents, containing 10,593 and 6,252 training sentences respectively. The labels were heavily imbalanced, with RatioOfTheDecision and Fact accounting for nearly half the data. I therefore used inverse-frequency loss weighting. Only 1.7% of sentences were truncated by the 128-token limit.
- **Phase 4:** I trained on a random 50% of the CJPE training judgments, amounting to 18,592 documents. The documents were extremely long, with a median length of approximately 2,500 tokens. I used 512-token chunks with 64-token overlap and retained at most 16 chunks per document. The dev set was used for threshold tuning, which improved macro-F1 from 0.73 to 0.78.
- **Phase 6:** The dataset contained 7,030 training and 100 test documents. Documents had a median length of approximately 3,100 words, while reference summaries were around 640 words. This motivated the use of semantic chunking and two-stage summarization. Due to computational limitations, I trained on a random 10% of the training data. The generated summaries were shorter than the references, partly because of the output-length limit.
- **Phase 7:** The training, development and test splits contained 827, 118 and 237 queries respectively. Each query had approximately 6–8 relevant cases, while the model was restricted to returning seven results. I fine-tuned on a random 40% of the usable training queries due to computational constraints.

Table 2: Phase-1 corpus distribution

Source	**Proportion**
Legal Acts	10%
High Court judgments	40%
Supreme Court judgments	40%
Tribunal documents	10%

3.3 Preprocessing

I used a custom byte-level BPE tokenizer with a 30K vocabulary, trained from scratch on approximately 5M tokens sampled from the four Phase-1 sources. Text was split on whitespace, converted

into UTF-8 bytes with an end-of-word marker, and processed using the learned merges. I did not lowercase or normalize the text.

For Phase 2, character offsets were reconstructed manually since the tokenizer did not provide offset mappings. Punctuation was separated from word boundaries to improve alignment with gold spans. Documents were processed in 512-token windows with a stride of 128, and each window was responsible only for its middle region during prediction.

For Phase 3, the provided sentence segmentation was retained, with sentences truncated at 128 tokens and processed in batches of 32. Phase 4 used 512-token chunks with 64-token overlap and a maximum of 16 evenly distributed chunks per document. Phase 5 used up to four evenly spaced chunks of 1024 tokens.

For Phase 6, sentences were split using a regex that accounted for legal abbreviations, followed by semantic chunking into approximately 650-token sections with two-sentence overlap. Finally, Phase 7 used the same preprocessing pipeline for queries and candidate cases, converting each case into a fixed-length fingerprint and then into a single vector.

Table 3: Preprocessing configurations across phases

Phase	**Chunk length**	**Additional setting**
1	1024	Fixed-length blocks
2	512	Stride 128
3	128	Sentence-level processing
4	512	Overlap 64, max. 16 chunks
5	1024	Max. 4 chunks
6	650	Semantic chunking
7	Fixed budget	Case fingerprints

4 Approach and Model Evolution

Architecture and tokenizer. I built a decoder-only Transformer from scratch with 12 pre-norm layers, 12 attention heads, hidden size 768, FFN dimension 3072, GELU activation and a context length of 1024. It uses RoPE, tied input-output embeddings and no dropout. A sigmoid gate applied to concatenated attention heads performed slightly better than the ungated version. The final model has 115.14M parameters.

I trained a custom byte-level BPE tokenizer with a vocabulary of 30,000 tokens on approximately 5M legal corpus tokens. Text is split on whitespace, converted into UTF-8 bytes and processed using learned merges with an end-of-word marker, without lowercasing or normalization.

Training and decoding. The model is trained using next-token cross-entropy on non-overlapping 1024-token blocks, with a 10/40/40/10 distribution across Acts, Supreme Court, High Courts and Tribunals. Training covered approximately 398M tokens for one epoch, with 2M tokens reserved for validation. I used AdamW, a learning rate of 3×10^{-4}, weight decay 0.1, 500 warm-up steps and cosine decay in FP16. Training took approximately 9 hours on a T4 GPU, reaching a validation loss of 1.990.

Generation uses beam search with 4 beams, temperature 0.3 and a no-repeat 3-gram constraint. Outputs target 128 words, with a maximum of 384 generated tokens. Batched generation uses left-padded prompts and KV caching.

Initial optimizations. Generation time was reduced from around 10 minutes per batch of 32 prompts to 78 seconds using KV caching, and then to 8 seconds through parallel batching. Early stopping based on word count brought it below a second. Streaming the training JSON instead of loading it entirely into memory also resolved Kaggle out-of-memory issues.

4.1 Transfer protocol (Phases 2–7)

All Phase-1 weights were uploaded to Hugging Face and reused to initialize subsequent task-specific models. Instead of training independent models from scratch, I added task-specific components and experimented with full and partial fine-tuning, LoRA, LoRMA, QLoRA and adapters. All six systems are fine-tuned descendants of the same foundation model.

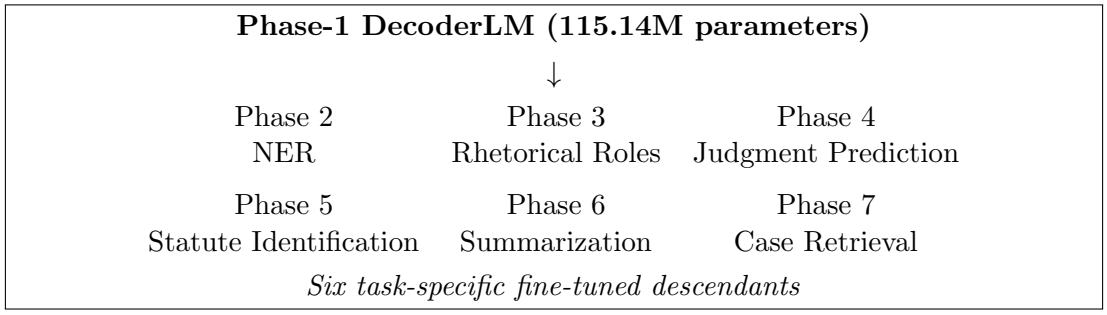


Figure 1: Transfer of the Phase-1 foundation model to subsequent tasks.

4.2 Phase 2 – Legal NER

I formulate NER as BIO token classification with 12 entity types and 25 tags, including O. The Phase-1 DecoderLM is followed by a bidirectional GRU and classification head. Character offsets are reconstructed manually from BPE tokens, with punctuation separated at word boundaries for better span alignment. Documents are processed in 512-token windows with a stride of 384, assigning predictions to the middle of each window to avoid overlap conflicts. I use square-root inverse-frequency class weights and retain 30% of empty windows. The final model uses full fine-tuning for 15 epochs, followed by argmax decoding and span reconstruction.

4.3 Phase 3 – Rhetorical role prediction

The Phase-1 DecoderLM encodes sentences, with mean pooling and the last-token representation concatenated and passed through a BiGRU to capture surrounding sentence context, followed by a classification head. Corporate Law and IT data are trained jointly. I experimented with LoRA, LoRMA, RS-LoRA, AdaLoRA, IA3, QLoRA, prompt tuning and knowledge distillation. The final configuration uses LoRMA with a BiGRU and a 12-layer student model trained using cross-entropy, KL divergence and cosine similarity losses.

4.4 Phase 4 – Court judgment prediction

This binary classification task uses 512-token chunks with 64-token overlap, retaining at most 16 evenly distributed chunks, including the beginning and end. The Phase-1 model encodes each chunk, and mean-pooled token representations are combined through gated attention pooling into a document embedding for binary classification. Training uses weighted cross-entropy, with the

prediction threshold tuned on development Macro-F1. After comparing fine-tuning strategies, full fine-tuning was selected, with experiments using 10% and 30% of the training data.

4.5 Phase 5 – Legal statute identification

This multi-label task predicts applicable statutes for each document. Documents are divided into 1024-token chunks, retaining up to four evenly spaced chunks. Masked mean pooling produces chunk representations, which are averaged into a document embedding and passed to a multi-label classification head with a separate binary classifier for each statute. Training uses one-hot cross-entropy, with a global threshold tuned on validation Macro-F1 and at least one statute predicted per document. I compared LoRMA (rank 16, alpha 32), partial fine-tuning and full fine-tuning, initially using 10% of the training data.

4.6 Phase 6 – Abstractive summarization

Due to the length of legal judgments, I use a two-stage summarization pipeline. Sentences are grouped into overlapping semantic chunks of up to 650 tokens, with topic shifts identified using TF-IDF cosine similarity. Individual chunks are summarized first, after which their summaries are combined to generate the final summary. Reference-summary sentences are aligned with chunks using TF-IDF similarity, and duplicate sentences from overlapping chunks are removed. I use LoRMA with rank 32 and alpha 64 on attention and feed-forward projections, comparing it against full fine-tuning, knowledge distillation, Pfeiffer and Houlsby adapters, prompt tuning and QLoRA. Generation uses beam search, a no-repeat n-gram constraint and an explicit end-of-summary marker.

4.7 Phase 7 – Prior Case Retrieval

The objective is to retrieve the seven most relevant cases for a query. I use semantic chunking and the Phase-6 summarization model to generate compact case fingerprints through hierarchical summarization. Queries and candidate cases are encoded using a shared Phase-1-based bi-encoder, with mean pooling and L2 normalization. Training uses contrastive learning with relevant cases as positives and TF-IDF-selected hard negatives. LoRMA adapters (rank 16, alpha 16) are applied to the query, key, value and output projections. I also experimented with combining embedding and TF-IDF similarity, tuning their relative contributions on the validation set before retrieving the top seven candidates.

5 Experiments and Implementation Details

5.1 Compute and training protocol

All experiments were conducted on Kaggle using GPU acceleration and CUDA. Gradient accumulation and activation checkpointing were used where necessary to manage GPU memory. Most experiments used AdamW with warmup and linear or cosine learning-rate decay, along with gradient clipping (maximum norm 1.0) and weight decay of 0.01 where applicable. LoRA/LoRMA experiments used separate learning rates for adapters and classification heads. Checkpoints were selected based on validation performance and the primary evaluation metric for each phase, rather than simply using the final epoch. Early stopping varied across phases, with some experiments using fixed epochs and others retaining the best checkpoint.

Table 4: Hyperparameters of the final model in each phase.

Hyperparameter	P1	P2	P3	P4	P5	P6	P7
Parameters (M)	115.1	115.1+	115.1+	115.34	115.1+	121.0	122.2
Context length	1024	512	128	512	1024	1024	640/512
Peak LR	3e-4	5e-4	3e-4	5e-4	1e-3	3e-4	2e-4
Batch × accum.	4×2	8 × 1	2 × 2	4 × 4	2 × 4	4 × 4	4 × 1
Epochs / steps	1	15	10	3	6	2	3
Dropout	0	0.1	0.2	0.1	0	0.05	0.0
Decoding / threshold	Beam 4	Argmax	Argmax	0.15	0.15	Beam 3	Top-7

5.2 Local evaluation

- I created fixed held-out validation splits from the available training data wherever an official development set was unavailable, using a fixed random seed for reproducibility. Wherever official train/dev splits were provided, I used them directly.
- Local and Codabench scores sometimes differed because they were evaluated on different sets of documents with potentially different distributions and difficulty levels. I therefore used local scores for hyperparameter experimentation and format checking mostly and Codabench scores for valid experiments.

5.3 Ablations and controlled comparisons

In Phase 1, I experimented with different tokenizer construction strategies, including frequency-based and NPMI-IDF based BPE. I also tested different dataset distributions for tokenizer training (0.8152 vs. 0.8346). At the model level, we compared sinusoidal and rotary positional encodings, encoder-decoder and decoder-only architectures, and the effect of gated attention (0.7796, 0.8152, 0.8346, 0.8228). I additionally investigated the effect of increasing the training data from 150M to 400M tokens (The 400M tokens model was not submitted during phase 1 but used as base model for fine tuning for subsequent phases). These experiments helped, identify the tokenizer and architectural configurations that performed best. Several computational optimizations were also implemented, including KV caching, parallel batch processing, early stopping during generation and streaming the dataset, reducing the time per batch from approximately 10 minutes to less than a second.

Across the remaining phases, we compared different fine-tuning strategies, including LoRA, LoRMA, full and partial fine-tuning, knowledge distillation, QLoRA and various adapter methods. We also investigated the contribution of BiGRU layers, different Phase-1 checkpoints, data scaling and semantic chunking. In Phase 2, adding a BiGRU improved the score to 0.4423, while full fine-tuning with the BiGRU achieved 0.4766 (0.5234 with more epochs), our best result in that phase. Both positive and negative results were considered while selecting the final configurations.

6 Results

Table 5: Per-phase variant comparison

Phase	Variant	Codabench primary metric
1	Encoder Decoder	0.8228
1	Gated Attention	0.8346
2	LMORA + BiGRU	0.4423
2	Full Fine Tuning + BiGRU	0.5234
3	LORMA	0.4623
3	Knowledge Distillation	0.4355
4	LORMA	0.619
4	Full Fine Tuning	0.6418
5	LOMRA	0.2860
5	Full Fine Tuning	0.2967
6	LORMA	0.8438
6	Full Fine Tuning	0.8385
7	QLORA	0.3662
7	AdaLORA	0.3674

7 Error Analysis and Discussion

7.1 Phase-wise failure modes

The major challenges occurred during Phase 1, where several optimizations were required to make training and inference feasible under limited computational resources. KV caching, parallel batch processing and truncation of generated tokens reduced generation time from around 10 minutes to less than a second per batch. Streaming the training data resolved memory issues. A tokenizer bug involving an ignored “ end character was also identified and fixed, significantly improving the score.

In Phase 2, the main challenge was entity identification and boundary prediction. LoRA and knowledge distillation performed relatively poorly, while introducing a BiGRU and switching to full fine-tuning significantly improved results.

In Phase 3, several fine-tuning strategies, including LoRA, LoRMA, AdaLoRA, QLoRA, IA3, prompt tuning and knowledge distillation, were explored alongside different Phase-1 checkpoints to identify the most effective configuration.

In Phase 4, limited training data made fine-tuning strategy selection challenging. Full fine-tuning performed better than the alternatives on 10% of the dataset, motivating experiments with larger data fractions and alternative architectures.

In Phase 6, generating coherent summaries from long judgments was the primary challenge. Semantic chunking and hierarchical summarization were introduced, alongside experiments with different fine-tuning strategies to improve results.

In Phase 7, the main challenge was effective retrieval from long legal documents. Semantic chunking, knowledge distillation and different similarity-based retrieval strategies were explored to improve performance.

7.2 What the metrics do not measure

ROUGE-L and BERTScore measure similarity with reference summaries rather than actual legal correctness or factual reliability. Similarly, span-F1 penalizes even slightly incorrect entity boundaries, without necessarily reflecting whether the entity itself was identified correctly. Therefore, good evaluation scores do not guarantee reliable legal outputs. In an actual law firm deployment, human verification would still be essential before using the model's predictions or summaries.

7.3 Limitations and what you would do next

The main limitations were limited computational resources, time. Additionally, errors from one phase could propagate to subsequent phases if the models were combined into a single pipeline. With more resources, I would experiment with longer context windows, incorporate additional legal datasets, evaluate outputs with legal experts and investigate joint training of multiple phases to reduce error propagation. "‘

7.4 Surprising findings

The gated attention mechanism which was used in the final implementation was a completely new idea, which upon testing gave slightly better results. This was unexpected as it was something I devised of on my own inspired from GRUs, and it wasn't tested before.

8 Conclusion

- The most significant improvements came from correcting the tokenizer implementation, introducing a BiGRU in Phase 2, and using full fine-tuning instead of parameter-efficient methods. On the other hand, knowledge distillation and several adapter-based approaches did not consistently provide better results.
- The biggest takeaway was that training a compact legal LM under strict computational and data constraints requires a lot of experimentation and careful optimization. Architectural complexity alone does not guarantee better performance, and relatively small implementation changes can make a surprisingly large difference.

LLM Usage Declaration

Q1. Which Large Language Model(s), if any, did you use while completing the competition or writing this report? Give model name and vendor (e.g., none / ChatGPT-4.x / Gemini x / Claude x / Grok / Copilot / other).

Answer: Gemini Pro

Q2. For each row, mark Yes or No and state the purpose in one short phrase. "Yes" is acceptable only for the uses permitted by the capstone document.

Part	Used LLM?	Purpose of using the LLM
Understanding concepts / papers	Yes	Couple of things seemed hard to grasp by simply reading their papers, using LLMs to understand those topics was convineint and very efficient as well
Ideation / brainstorming at a high level	Yes	LLMs were used for initial ideation while creating the first draft for some phases, but the ideation for variations were not using LLMs
Debugging small ancillary snippets	Yes	Debugging and writting ancilliary code and sometimes base template
Plotting / visualization code	Yes	This also comes under ancilliary code writting
Grammar / writing style of the report	Yes	Chekking grammatical mistakes throughout the report and formatting it
Report structure / section wording	No	
Designing the model architecture	No	
Hyperparameter search / tuning	No	
Analysis of results / conclusions	No	
Writing the core training / inference code	No	
Other (specify)	Yes	LLMs were used to sometimes generate the base template for phases.

Q3. Did you refer to sources other than lecture slides and an LLM? If yes, list them (papers, blogs, GitHub repositories, textbooks, official starter notebook, evaluation scripts). These must also appear in the bibliography when they influenced the method.

Answer:
Yes, the starter notebook for the project and boilerplate codes mentioned in the lecture notes were used.

Q4. Apart from any permitted LLM use, what was *your* contribution? In 8–12 sentences describe your own understanding, architecture decisions, implementation, experiments, and analysis. Name at least two things you tried that failed and what you concluded from them.
LLMs were used for verification purposes and, sometimes generating the base template as permitted by the professor, but core model architecture and design choices were always mine. Gating mechanism in Attention was my own idea inspired from GRUs. LLMs are incapable (atleast as of now) of suggesting new ideas that haven't been implemented and tried. Which is why, this makes it a distinguishing point. Apart from this, throughout the phases, I ensured that whatever code was being generated by LLMs, I understood it's purpose and its structure to some extent and the use of LLMs was restricted to minimal specially for writing the core architecture. LLMs were also often used to understand underlying concepts in more depth. Two things I tried but failed include NPMI-IDF based BPE Tokenizer and encoder decoder type model instead of simply decoder type.

References

[1] A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[2] J. Su et al., "RoFormer: Enhanced Transformer with Rotary Position Embedding," arXiv preprint arXiv:2104.09864, 2021.

[3] R. Sennrich, B. Haddow, and A. Birch, "Neural Machine Translation of Rare Words with Subword Units," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, 2016, pp. 1715–1725.

[4] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in *International Conference on Learning Representations (ICLR)*, 2019.

[5] E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," in *International Conference on Learning Representations (ICLR)*, 2022.

[6] T. Dettmers et al., "QLoRA: Efficient Finetuning of Quantized LLMs," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[7] N. Houlsby et al., "Parameter-Efficient Transfer Learning for NLP," in *Proceedings of the 36th International Conference on Machine Learning*, 2019, pp. 2790–2799.

[8] J. Pfeiffer et al., "AdapterFusion: Non-Destructive Task Composition for Transfer Learning," in *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*, 2021, pp. 487–503.

[9] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," arXiv preprint arXiv:1503.02531, 2015.

[10] T. Zhang et al., "BERTScore: Evaluating Text Generation with BERT," in *International Conference on Learning Representations (ICLR)*, 2020.

[11] C.-Y. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," in *Text Summarization Branches Out*, 2004, pp. 74–81.

[12] Starter Code for the Project Phase 1

[13] Boiler Plate code mentioned in Lecture Notes